NBA Analyst Agent

A natural-language analytics agent over NBA box-score data. A user asks a basketball question in plain English; the agent looks up the schema, writes SQL against CSV-backed tables, checks the result, and answers in plain English with the numbers and the filters it used.

Why this project: it is a small, fully measurable version of the "ask questions of our business data" problem. Swap the NBA CSVs for corporate extracts and the same architecture, evaluation method, and failure modes apply.

Start here: what to do next in Colab

You have a Kaggle dataset (CSV files) in Colab and API keys for OpenAI and Gemini. Do these in order. Each step has a "done when" check so you know it worked.

#	Do this	Done when
1	Open `notebooks/colab_runner.ipynb` in Colab. Add `OPENAI_API_KEY` and `GEMINI_API_KEY` under **Secrets** (key icon, left sidebar) and switch on notebook access. Run the clone/install cell.	`python src/list_models.py` prints model names for both providers
2	Find the folder where the Kaggle files are (look in the Colab file browser; often `/content/...`). Run `python src/inspect_csvs.py <that folder>`.	You can see every CSV, its columns, and 3 sample rows
3	Decide which CSV holds **one row per player per game** (box scores). Copy `data_mapping.example.json` to `data_mapping.json` and edit it so each *source column* maps to our *column name* (see section 3).	The file lists your CSV name and renames for every stat
4	Run `python src/prepare_data.py --src <folder> --mapping data_mapping.json`. Read every `NOTE:` line it prints.	It prints row counts and writes `data/player_games.csv` and `data/team_games.csv`
5	Run `python -m pytest tests -q`, then `python src/evaluate.py --check-gold`. Compare 3–4 gold answers with Basketball-Reference.	Gold answers look right; skipped questions are ones your data cannot answer
6	Ask the agent one question by hand with each provider (notebook step 4).	You get an answer plus the SQL it ran
7	`python src/evaluate.py --provider openai --limit 5 --judge`, check the cost in the provider dashboard, then run the full set for each provider.	You have `results/eval_*.csv` for both providers
8	Read the failures, pick one fix (section 5), rerun. Repeat. Gemma comes after this works.	A before/after accuracy table

About SQL in Colab: you do not need a database. DuckDB is the SQL engine here: it installs with one `pip install` (already in `requirements.txt`), runs inside the notebook, and queries your CSV files directly. Your data stays as CSVs.

If step 2 or 3 gets confusing, paste the output of `inspect_csvs.py` to your helper and ask for the mapping file.

1. Goals and non-goals

Goals

1. Answer factual and analytical basketball questions correctly from data, with no invented statistics.
2. Say "the data can't answer that" when it can't (salaries, injuries, seasons outside the data).
3. Measure accuracy with a fixed test set, then improve it and show the before/after.
4. Document what transfers to a business setting.

Non-goals: training an LLM from scratch, live scores, predictions, play-by-play or tracking analysis.

Design principle: the model never *remembers* statistics. It *finds* them. Numbers come only from query results, which is what makes the answers checkable.

2. Architecture

```
user question
     |
     v
+-----------+   get_schema()    +--------------------------+
|   LLM     | ----------------> | data dictionary + live   |
| (OpenAI   |   run_sql(sql)    | schema                   |
| or Gemini)| ----------------> +--------------------------+
|           | <---------------- DuckDB (in-memory, read-only)
+-----------+   rows as CSV      tables loaded from data/*.csv
     |                           views: player_season, team_season
     v
plain-English answer + filters used
```

Piece	File	Job
Data prep	`src/inspect_csvs.py`, `src/prepare_data.py`, `data_mapping.example.json`	Look inside a Kaggle dataset, then convert it into the two tables the agent needs

Piece	File	Job
Data pull (alternative)	`src/pull_data.py`	Downloads box scores via `nba_api` instead of Kaggle (often blocked in Colab)
Database	`src/db.py`	Loads CSVs into DuckDB, defines season views, guards SQL
Agent	`src/agent.py`	System prompt, two tools, and the question-to-answer flow
LLM layer	`src/llm.py`	One tool-calling loop each for OpenAI (Responses API) and Gemini (`google-genai`), with retry on rate limits
Eval	`src/evaluate.py`, `eval/questions.csv`	32 questions with gold SQL; automatic + LLM-judge grading
UI	`src/app.py`	Optional Gradio chat
Models	`src/list_models.py`	Lists the models your API keys can use
Docs	`docs/data_dictionary.md`	Column meanings and query gotchas; the agent reads this
Tests	`tests/`	Offline tests on fake data (pipeline, SQL guard, gold SQL, grader)

Why DuckDB and not SQLite: DuckDB reads CSVs directly, needs no database server or file, and runs in Colab with one `pip install`. The data stays CSV; SQL is just the query language.

Why views: `player_season` and `team_season` pre-compute per-game averages, shooting percentages, true shooting, wins, and opponent points. Without them the model re-derives these every time, and that is where most wrong answers come from. It is the same reason business teams build a semantic layer.

Safety guards (`db.py`): one statement only, `SELECT` only, external file access disabled after load, configuration locked, 20 second timeout, 50-row result cap.

3. Data

The agent needs two tables, both **one row per game**:

File	Grain	Needed for
`data/player_games.csv`	player x game (box scores)	player stats, leaderboards, game-level questions
`data/team_games.csv`	team x game	team records, points allowed. Can be derived from player rows

Full column list and meanings: `docs/data_dictionary.md`.

Using your Kaggle dataset

Kaggle datasets all name their columns differently, so there is a one-time conversion step.

Step A. Inspect. `python src/inspect_csvs.py /path/to/kaggle/folder` shows each CSV's columns and sample rows. Find the file with one row per player per game.

Step B. Write the mapping. Copy `data_mapping.example.json` to `data_mapping.json`. The example uses made-up source column names; replace the left side of each pair with your dataset's real column names:

```
        {
          "season_year_is": "start",
          "player_games": {
            "file": "your_box_scores.csv",
            "rename": { "<your column>": "<our column>", "PLAYER_NAME": "player_name",
"PTS": "pts" },
            "defaults": { "season_type": "Regular Season" }
          },
          "team_games": "derive"
        }
```

Our column names that **must** exist after mapping (the agent's views cannot work without them): `season, season_type, player_id, player_name, team_abbreviation, game_id, game_date, wl, min, fgm, fga, fg3m, fg3a, ftm, fta, reb, ast, stl, blk, tov, pts`. Nice to have (filled with empty values if missing, and the script tells you): `oreb, dreb, pf, plus_minus, matchup, team_id, team_name`.

What the script fixes for you: dates become `YYYY-MM-DD`; `season` is derived from the date if there is no season column; minutes like `35:12` become numbers; win flags (`1/0/True/False`) become `W`/`L`; `reb` is computed from `oreb + dreb` if missing. Use `"season_year_is": "end"` if the dataset labels a season by the year it ends. Use `defaults` for constants, such as `season_type` when the file only has regular season games. If the dataset has a real team game file, give it its own block instead of `"derive"`.

Step C. Convert and check.

```
        python src/prepare_data.py --src /path/to/kaggle/folder --mapping
data_mapping.json
        python src/evaluate.py --check-gold
```

Read the `NOTE:` lines. Typical ones: no `matchup` column (the home/away question will be skipped), no `team_name` (questions that use full team names such as "Celtics" will be skipped), games without exactly two team rows (opponent points will be off for those games). `evaluate.py` automatically skips questions whose gold SQL cannot run on your data or returns no rows (for example a player or season that is not in your dataset), and tells you which.

Edit the eval for your data. The sample questions use 2021-22 through 2025-26 and well-known players. If your dataset covers different seasons, open `eval/questions.csv` and change the seasons and names so most questions apply. Also update the coverage statement in your write-up: which seasons, whether playoffs are included, and any missing stats. Check the dataset's license on Kaggle before sharing it.

Alternative: download from the NBA directly

`python src/pull_data.py` uses `nba_api` to fetch `stats.nba.com`. It often times out from cloud IPs like Colab, so if you use it, run it on your laptop and upload the `data/` folder. Options: `--seasons 2023-24 2024-25`, `--proxy <url>`. Downloads are cached in `data/raw/` and resume after a failure.

4. Setup (Colab)

1. Open `notebooks/colab_runner.ipynb` in Colab. Its first cell clones this repo.
2. Add `OPENAI_API_KEY` and `GEMINI_API_KEY` under Colab **Secrets** and enable notebook access for each.
3. Run `python src/list_models.py` and set `--model` (or `NBA_OPENAI_MODEL` / `NBA_GEMINI_MODEL`) to models your keys can actually use. The defaults in `src/llm.py` are only starting points.
4. Do the data steps in "Start here" (inspect, map, prepare), then the checks and evaluation.

Locally:

```
        pip install -r requirements.txt
        export OPENAI_API_KEY=...
        export GEMINI_API_KEY=...
        python src/list_models.py
        python src/inspect_csvs.py /path/to/kaggle/folder
        python src/prepare_data.py --src /path/to/kaggle/folder --mapping
data_mapping.json
        python -m pytest tests -q
        NBA_PROVIDER=gemini python src/agent.py "Who led the league in assists per game
in 2024-25?"
        python src/evaluate.py --provider openai --judge
```

5. Evaluation

`eval/questions.csv` has 32 questions in 8 categories: lookup, leaderboard, team, comparison, trend, game-level, playoffs, derived, and **unanswerable** (the agent must decline, not invent). Each answerable question has a gold SQL query. The harness runs the gold SQL against your real data, so the expected answers stay correct whichever seasons you pull.

Two graders:

- **auto**: every value in the gold result must appear in the answer (numbers within rounding).
- **judge** (`--judge`): a model compares the answer with the gold result. More forgiving of phrasing. By default the judge is the *other* provider than the agent, to avoid a model favoring its own family.

Both have blind spots, so read a sample of failures by hand. Failures are recorded in `results/eval_*.csv` with the queries the agent ran, so you can tag each as: wrong column, bad join, missing filter, misread question, hallucinated number, or grader error.

Experiments that make a good write-up (change one thing at a time, rerun the full set):

1. Baseline: no `get_schema` description, no views (query raw tables only).
2. Add the data dictionary and views (the current design).
3. Add 5–10 worked question-to-SQL examples to the system prompt.
4. Model comparison: OpenAI vs Gemini, and a small vs larger model from each (`--provider`, `--model`); plot accuracy vs cost and latency.
5. Expand the question set with your own questions, especially the ones you tried that failed.

Report accuracy overall and per category, and the failure-type breakdown. A statement like "view and dictionary changes moved accuracy from X% to Y% and removed most wrong-column errors" is the result worth presenting.

> The gold SQL for each question was checked for validity against the schema on synthetic data. It has **not** yet been checked against real NBA data. After preparing your data, run `python src/evaluate.py --check-gold` and sanity check a few results against Basketball-Reference before trusting any score.

6. Build plan

Phase	Work	Done when
1. Data	Inspect the Kaggle files, write the mapping, run `prepare_data.py`, run tests, skim for oddities (missing seasons, duplicate rows)	`--check-gold` returns sensible numbers
2. Agent v1	Run `agent.py` on 10 questions by hand	End-to-end answers with queries shown
3. Measure	Full eval, categorize failures	Baseline accuracy recorded
4. Improve	Experiments in section 5, one change at a time	Before/after table
5. Present	UI demo plus results and lessons	Short deck or notebook walkthrough

7. Mapping to your business work

Here	At work
`player_games` / `team_games` CSVs	Fact tables or exports
`player_season` / `team_season` views	Governed metric definitions (a semantic layer)
`data_dictionary.md`	Metric glossary, the single most valuable artifact
Min-games filter, `season_type` filter	Business rules the agent must apply every time
Eval set with gold SQL	Regression tests you rerun on every prompt or model change
"Data can't answer that" cases	Trust: an agent that refuses beats one that guesses
SQL guard	Read-only access, row limits, timeouts for any real data source

8. Risks and limitations

- **Wrong-but-plausible answers** are the main risk. Mitigation: views, dictionary, eval, showing the queries.
- **Ambiguity** ("best player", "last season") needs a stated assumption. The prompt requires the agent to name its filters.
- **Cost and rate limits**: the full eval makes a few hundred API calls. Run `--limit 5` first and check usage in each provider's dashboard before scaling up. Free tiers return 429 errors under load; `llm.py` backs off and retries automatically.
- **Stateless**: each question starts fresh, so follow-ups like "what about Embiid?" don't work yet.
- **Data freshness**: the Kaggle files are a snapshot. The agent states the date range it covers, and cannot answer about games after it.

9. Stretch ideas

- Multi-turn memory for follow-up questions.
- A chart tool (matplotlib) so the agent can answer "show me ..." questions.
- A "scouting report" tool that combines several queries.
- Retrieval over articles or notes for qualitative context.
- Fine-tune a small open model (e.g. 1B–3B, LoRA) on synthetic question-to-SQL pairs for this schema and compare it with the API model on your eval. This is the "build a model" angle if your cohort needs one.

Appendix A: Data dictionary

Source: `nba_api` (`LeagueGameLog` endpoint). Every table is loaded into DuckDB from `data/*.csv`. Box-score stats are per game unless a column name says otherwise.

Conventions

- `season` is text like `2024-25`. `season_type` is `Regular Season` or `Playoffs`. **Always filter `season_type`.**
- `game_date` is `YYYY-MM-DD`. `game_id` is shared by the two teams (and all players) in a game.
- `matchup`: `BOS vs. NYK` means BOS played at home; `BOS @ NYK` means BOS played away.
- `wl` is `W` or `L` for the row's team. `min` is minutes played.
- A player traded mid-season has several `team_abbreviation` values; the season views combine them.
- Not in this data: salaries, injuries, play-by-play, tracking, advanced defensive metrics, draft info.

Tables (one row per ...)

player_games (player per game)

season, season_type, season_id, player_id, player_name, team_id, team_abbreviation, team_name, game_id, game_date, matchup, wl, min, fgm, fga, fg_pct, fg3m, fg3a, fg3_pct, ftm, fta, ft_pct, oreb, dreb, reb, ast, stl, blk, tov, pf, pts, plus_minus, fantasy_pts

team_games (team per game)

Same box-score columns as player_games but for the whole team: season, season_type, team_id, team_abbreviation, team_name, game_id, game_date, matchup, wl, min, fgm ... pts, plus_minus.

teams (team)

id, full_name, abbreviation, nickname, city, state, year_founded

players (player directory, optional)

person_id, display_first_last, rosterstatus, from_year, to_year, team_id, team_abbreviation, ...

Views (pre-aggregated, prefer these for season questions)

player_season (player per season per season_type)

player_id, player_name, season, season_type, teams, games, mpg, total_pts, ppg, total_reb, rpg, total_ast, apg, spg, bpg, tpg, fgm, fga, fg_pct, fg3m, fg3a, fg3_pct, ftm, fta, ft_pct, ts_pct (true

shooting = pts / (2 * (fga + 0.44 * fta))), avg_plus_minus. Percentages are fractions (0.452 = 45.2%).

team_season (team per season per season_type)

team_id, team_name, team_abbreviation, season, season_type, games, wins, losses, win_pct, ppg, opp_ppg (points allowed per game), avg_margin (ppg - opp_ppg), fg3m_pg, fg3_pct, rpg, apg, tpg.

Gotchas for query writing

- Leaderboards need a minimum-games filter (`games >= 40` is a reasonable default), otherwise a player with 2 games tops every percentage list.
- "Last season" means the most recent complete season in the data, not the calendar year.
- Per-game averages in `player_season` are the mean over games played, not over team games.
- Back-to-backs, home/away splits, and win/loss splits come from `player_games` / `team_games`.

Appendix B: Evaluation questions

#	Category	Question
1	lookup	How many points per game did Stephen Curry average in the 2023-24 regular season?
2	lookup	How many games did LeBron James play in the 2023-24 regular season?
3	lookup	What was Nikola Jokić's true shooting percentage in the 2024-25 regular season?
4	lookup	How many total rebounds did Domantas Sabonis grab in the 2023-24 regular season?
5	lookup	What was Jayson Tatum's three-point percentage in the 2023-24 regular season?
6	leaderboard	Who led the league in points per game in the 2023-24 regular season (minimum 50 games)?
7	leaderboard	Who were the top 3 in assists per game in the 2024-25 regular season (minimum 50 games)?
8	leaderboard	Who made the most three-pointers in the 2023-24 regular season?
9	leaderboard	Among players with at least 50 games and 15 points per game in 2023-24, who had the best true shooting percentage?
10	leaderboard	Who led the league in blocks per game in 2024-25 (minimum 50 games)?
11	team	Which team had the most wins in the 2023-24 regular season?
12	team	What was the Boston Celtics' average point differential in the 2023-24 regular season?
13	team	Which team allowed the fewest points per game in the 2024-25 regular season?
14	team	How many regular season wins did the Oklahoma City Thunder have in 2024-25?
15	team	Which team made the most threes per game in the 2023-24 regular season?
16	comparison	Compare Luka Dončić and Shai Gilgeous-Alexander in points, rebounds, and assists per game in the 2024-25 regular season.
17	comparison	Who scored more total points in the 2023-24 regular season, Kevin Durant or Devin Booker?
18	comparison	Who averaged more points per game in the 2023-24 regular season, Nikola Jokić or Joel Embiid?
19	trend	How did Stephen Curry's points per game change each regular season from 2021-22 to 2024-25?
20	trend	Which team improved its win total the most from the 2022-23 to the 2023-24 regular season?
21	game-level	What is the most points Luka Dončić scored in a single regular season game in 2023-24?
22	game-level	How many 40-point games did Devin Booker have in the 2023-24 regular season?

#	Category	Question
23	game-level	How many triple-doubles did Nikola Jokić have in the 2023-24 regular season?
24	game-level	What were Stephen Curry's points per game at home versus on the road in the 2023-24 regular season?
25	game-level	What was the Lakers' win-loss record in 2023-24 regular season games that LeBron James did not play in?
26	playoffs	Who led all players in points per game in the 2023-24 playoffs (minimum 10 games)?
27	playoffs	Which team won the most playoff games in 2023-24?
28	derived	Which player had the best assist-to-turnover ratio in the 2023-24 regular season among players with at least 50 games and 5 assists per game?
29	derived	How many points per 36 minutes did Victor Wembanyama average in the 2024-25 regular season?
30	unanswerable	What is LeBron James' salary this season?
31	unanswerable	Who will win the 2027 MVP award?
32	unanswerable	How many points per game did Michael Jordan average in 1996-97?

Appendix C: Example data_mapping.json

The column names on the left are placeholders. Replace them with your dataset's real columns.

```
{
  "season_year_is": "start",
  "player_games": {
    "file": "player_box_scores.csv",
    "rename": {
      "personId": "player_id",
      "playerName": "player_name",
      "teamTricode": "team_abbreviation",
      "teamName": "team_name",
      "gameId": "game_id",
      "gameDate": "game_date",
      "win": "wl",
      "minutes": "min",
      "fieldGoalsMade": "fgm",
      "fieldGoalsAttempted": "fga",
      "threePointersMade": "fg3m",
      "threePointersAttempted": "fg3a",
      "freeThrowsMade": "ftm",
      "freeThrowsAttempted": "fta",
      "reboundsOffensive": "oreb",
      "reboundsDefensive": "dreb",
```

```
            "reboundsTotal": "reb",
            "assists": "ast",
            "steals": "stl",
            "blocks": "blk",
            "turnovers": "tov",
            "foulsPersonal": "pf",
            "points": "pts",
            "plusMinusPoints": "plus_minus"
          },
          "defaults": {"season_type": "Regular Season"}
        },
        "team_games": "derive"
      }
```